WARING DECOMPOSITION OF DERIVATIVES AND ITS APPLICATIONS

AUTHOR ONE* AND AUTHOR TWO†

Abstract. Computing high-order derivatives in neural network residuals is often computationally and memory intensive, primarily due to the large computational graphs caused by automatic differentiation. Within the Taylor-mode framework used by STDE, we introduce the Waring decomposition of derivatives (WDD) method, which constructs directional Taylor representations from power-sum decompositions of polynomial symbols. We prove that the minimum length of an exact complex directional Taylor representation of a target partial derivative equals the Waring rank of its corresponding monomial, and a classical roots-of-unity Waring decomposition attains this minimum. Experiments on individual high-order partial derivatives assess the runtime of the WDD method relative to the deterministic directional Taylor construction reviewed in STDE++. Numerical experiments show that the WDD method reduces derivative-evaluation time for several tested patterns. In PINN training for the KdV and KP-II equations, it attains comparable mean solution errors in less training time than nested coordinate Jacobian–vector products and the STDE++ construction, respectively, at equal update counts, with both interior and constraint training points resampled at every update.

Key words. high-order derivatives, Taylor-mode automatic differentiation, Waring decomposition, directional Taylor coefficients

AMS subject classifications. 65D25, 14N07

1. Introduction.

Physics-informed neural networks approximate solutions of differential equations by minimizing loss functions constructed from the corresponding PDE residuals. This requires derivatives of the network with respect to its inputs. For high-order equations, the repeated evaluation of these derivatives can dominate both the runtime and memory cost of residual evaluation.

Automatic differentiation is the standard tool for evaluating these derivatives [1, 9, 20]. Several exact approaches reduce its cost by exploiting the structure of the differential operator. For example, forward Laplacian methods propagate the Laplacian through the network without constructing the full Hessian [18]. Such methods can be efficient, but their differentiation rules are designed for specific structured operators and do not readily extend to general high-order partial derivatives. Taylor-mode automatic differentiation provides a more general alternative by propagating a truncated univariate Taylor series through the network [2, 9]. Existing general constructions recover multivariate high-order derivatives from multiple Taylor expansions, often by reconstructing the full derivative tensor [8]. The number of tensor components grows rapidly with both the input dimension and the derivative order, resulting in substantial computational and GPU-memory costs [24].

Randomization offers another route to reducing derivative costs. Stochastic dimension gradient descent uses coordinate sampling to evaluate only a subset of derivative terms during each optimization step [13]. Hutchinson-type estimators use random projections to approximate trace-type differential operators [11,14,21]. However, both approaches still rely on automatic differentiation to compute the sampled derivatives or projected vector products. A separate line of work avoids automatic differentiation entirely when computing derivatives. Finite differences approximate derivatives from nearby network evaluations [6,7], whereas randomized smoothing uses Stein's identity

*Institution One (author1@example.com).
†Institution Two (author2@example.com).

to express derivatives of a smoothed network as expectations of perturbed function values [10,28]. Although these methods reduce computational costs, their performance depends on the choice of method-specific parameters. Finite differences require a step size that balances truncation and round-off errors, randomized smoothing depends on both the smoothing bandwidth and the Monte Carlo sample size, and the randomized estimators require a sampling distribution and budget. For the stochastic methods, these choices directly affect estimator variance: insufficient or poorly matched sampling can yield high-variance residual estimates, while a small smoothing bandwidth can sharply amplify variance for high-order derivatives and may destabilize PINN training [10, 12, 25].

STDE evaluates derivative-tensor contractions through univariate Taylor-mode automatic differentiation and uses them to construct stochastic estimators of linear differential operators [24]. STDE++ also studies exact recovery of mixed partial derivatives, reviewing the directional Taylor construction of Griewank, Utke, and Walther [8] and developing constructions based on higher-order Taylor expansions along nonlinear curves [25].

Within the Taylor-mode framework used by STDE [24], we use Waring decompositions of polynomial symbols to construct directional representations of homogeneous constant-coefficient differential operators. We call this construction the Waring decomposition of derivatives (WDD) method and specialize it to mixed partial derivatives. Our main contributions are:

- We establish an exact correspondence between directional Taylor representations of homogeneous constant-coefficient differential operators and power-sum identities for their polynomial symbols. This correspondence identifies the minimum length of an exact complex directional Taylor representation with the Waring rank of the symbol.
- For a single mixed partial derivative, we combine the correspondence with the established complex Waring-rank and roots-of-unity results for monomials to obtain a closed-form complex directional Taylor representation of minimum length. A batched Taylor-mode implementation evaluates the resulting directional Taylor coefficients independently and in parallel.
- We compare the WDD method with the deterministic directional Taylor construction reviewed in STDE++ on individual high-order partial derivatives, examining runtime across derivative patterns without outer whole-function JIT compilation. KdV and KP-II PINN comparisons measure training time and solution error at equal update counts, with matched initialization and resampled interior and constraint-point sequences.

The remainder of the paper is organized as follows. Section 2 introduces the method of PINNs, multi-index notation, and directional Taylor coefficients. Section 3 establishes the operator–symbol correspondence, analyzes the computational complexity, specializes the correspondence to monomial Waring decompositions, and presents the resulting minimum-direction formula with its batched Taylor realization. Section 4 evaluates the derivative backend on individual high-order partial derivatives and examines its use in complete KdV and KP-II PINN training. Section 5 concludes. A coefficient verification of the cited roots-of-unity decomposition is given in Appendix A.

2. Preliminaries.

2.1. Method of PINNs. Let $\Omega \subset \mathbb{R}^d$ be a computational domain and $\Gamma \subseteq \partial\Omega$ the part of its boundary on which constraints are prescribed. In the numerical examples, Ω is the space–time domain, d includes the time coordinate, and Γ includes the initial surface and the prescribed spatial boundary surfaces. We consider the problem

$$\mathcal{A}u_\star = f \quad \text{in } \Omega, \qquad \mathcal{B}u_\star = g \quad \text{on } \Gamma, \tag{1}$$

where $\mathcal{A}$ is a differential operator and $\mathcal{B}$ is a boundary operator or a collection of boundary operators. In the PINN framework, $u_\star$ is approximated by a neural network u_θ [15, 17, 23, 27]. Let $\{\boldsymbol{x}_{r,i}\}_{i=1}^{n_r} \subset \Omega$ and $\{\boldsymbol{x}_{b,i}\}_{i=1}^{n_b} \subset \Gamma$ denote the residual and constraint training points. For weights $\lambda_r, \lambda_b > 0$, the network parameters are obtained by minimizing

$$\mathcal{J}(\theta) := \frac{\lambda_r}{n_r} \sum_{i=1}^{n_r} \left|\mathcal{A}u_\theta(\boldsymbol{x}_{r,i}) - f(\boldsymbol{x}_{r,i})\right|^2 + \frac{\lambda_b}{n_b} \sum_{i=1}^{n_b} \left|\mathcal{B}u_\theta(\boldsymbol{x}_{b,i}) - g(\boldsymbol{x}_{b,i})\right|^2. \tag{2}$$

Evaluating the PDE residual in (2) requires derivatives of u_θ. For high-order equations, nested automatic differentiation can dominate the time and memory required for residual evaluation because it retains large intermediate computational graphs and repeatedly differentiates through the network.

2.2. Multi-index notation. Let $\mathbb{N}_0$ denote the set of nonnegative integers. For each coordinate x_i, let ∂_i denote differentiation with respect to x_i. For a multi-index $\beta = (\beta_1, \beta_2, \ldots, \beta_d) \in \mathbb{N}_0^d$, write

$$|\beta| := \sum_{i=1}^{d} \beta_i, \qquad \beta! := \prod_{i=1}^{d} \beta_i!, \qquad \partial^\beta := \partial_1^{\beta_1} \partial_2^{\beta_2} \cdots \partial_d^{\beta_d}. \tag{3}$$

For $\boldsymbol{v} = (v_1, v_2, \ldots, v_d)^\top \in \mathbb{C}^d$, we also write $\boldsymbol{v}^\beta := v_1^{\beta_1} v_2^{\beta_2} \cdots v_d^{\beta_d}$. The zero multi-index is denoted by $\mathbf{0}$, so that $\partial^{\mathbf{0}} u = u$.

2.3. Directional Taylor coefficients. At each evaluation point $\boldsymbol{x}$, assume that u admits a holomorphic extension to an open neighborhood of $\boldsymbol{x}$ in $\mathbb{C}^d$, again denoted by u. For every integer $k \geq 1$ and every $\boldsymbol{v} \in \mathbb{C}^d$, define the order-$k$ directional Taylor coefficient by

$$T_k(\boldsymbol{x}; \boldsymbol{v}) := \frac{1}{k!} \frac{d^k}{d\tau^k} u(\boldsymbol{x} + \tau \boldsymbol{v}) \Big|_{\tau=0}. \tag{4}$$

Thus, $T_k(\boldsymbol{x}; \boldsymbol{v})$ is the coefficient of τ^k in the Taylor expansion of $u(\boldsymbol{x} + \tau\boldsymbol{v})$; we set $T_0(\boldsymbol{x}; \boldsymbol{v}) = u(\boldsymbol{x})$. The chain rule gives

$$T_k(\boldsymbol{x}; \boldsymbol{v}) = \frac{1}{k!} D^k u(\boldsymbol{x})[\boldsymbol{v}, \boldsymbol{v}, \ldots, \boldsymbol{v}], \tag{5}$$

where $D^k u(\boldsymbol{x})$ denotes the symmetric complex k-linear derivative of u at $\boldsymbol{x}$.

Writing $\boldsymbol{v} = (v_1, v_2, \ldots, v_d)^\top$, the multivariate Taylor formula [19, Sec. 3.17] gives

$$T_k(\boldsymbol{x}; \boldsymbol{v}) = \sum_{|\beta|=k} \frac{\boldsymbol{v}^\beta}{\beta!} \partial^\beta u(\boldsymbol{x}). \tag{6}$$

3. Polynomial symbols and directional representations. In this section, we first establish an exact correspondence between directional Taylor representations of homogeneous constant-coefficient differential operators and power-sum identities for their polynomial symbols. We next establish the existence of directional representations and analyze their computational cost. We then specialize the correspondence to a mixed partial derivative, whose monomial symbol permits a minimum-direction formula through established Waring-rank results.

3.1. Exact operator–symbol correspondence. Let

$$\mathcal{L} = \sum_{|\beta|=p} a_\beta \partial^\beta, \qquad a_\beta \in \mathbb{C}, \qquad p \geq 1, \tag{7}$$

be a nonzero homogeneous constant-coefficient differential operator. We seek an exact linear representation of $\mathcal{L}u$ by order-p directional Taylor coefficients within the Taylor-mode framework used by STDE [24]. The following proposition characterizes the required coefficients and directions through power-sum identities for the polynomial symbol of $\mathcal{L}$.

PROPOSITION 3.1 (Exact operator–symbol correspondence). *Let R be a positive integer, and let $c_r \in \mathbb{C}$ and $\boldsymbol{v}_r \in \mathbb{C}^d$ for $r = 1, 2, \ldots, R$. For formal polynomial variables $\boldsymbol{z} = (z_1, z_2, \ldots, z_d)^\top$, we denote the standard bilinear pairing by $\boldsymbol{v} \cdot \boldsymbol{z} := \sum_{j=1}^d v_j z_j$. Then the identity*

$$\mathcal{L}u(\boldsymbol{x}) = \sum_{r=1}^R c_r T_p(\boldsymbol{x}; \boldsymbol{v}_r) \tag{8}$$

holds for every u satisfying the assumption in Section 2.3 and every evaluation point $\boldsymbol{x}$ if and only if

$$\sum_{|\beta|=p} a_\beta \boldsymbol{z}^\beta = \frac{1}{p!} \sum_{r=1}^R c_r (\boldsymbol{v}_r \cdot \boldsymbol{z})^p \qquad \text{in } \mathbb{C}[z_1, z_2, \ldots, z_d]. \tag{9}$$

Proof. We show that both (8) and (9) are equivalent to

$$\sum_{r=1}^R c_r \boldsymbol{v}_r^\beta = \beta!\, a_\beta, \qquad \text{for every } \beta \in \mathbb{N}_0^d \text{ with } |\beta| = p. \tag{10}$$

(i) (10) $\iff$ (8)

We first prove that (10) implies (8). Substituting (6) into the right-hand side of (8) gives

$$\sum_{r=1}^R c_r T_p(\boldsymbol{x}; \boldsymbol{v}_r) = \sum_{|\beta|=p} \left(\frac{1}{\beta!} \sum_{r=1}^R c_r \boldsymbol{v}_r^\beta \right) \partial^\beta u(\boldsymbol{x}). \tag{11}$$

Using (10) in (11) yields (8).

Conversely, we prove that (8) implies (10). For any β with $|\beta| = p$, choose $u_\beta(\boldsymbol{y}) = (\boldsymbol{y} - \boldsymbol{x})^\beta$. Substituting u_β into (8) gives

$$\beta!\, a_\beta = \sum_{r=1}^R c_r \boldsymbol{v}_r^\beta.$$

Therefore, (8) and (10) are equivalent.

(ii) (9) $\iff$ (10)

The multinomial theorem gives

$$\frac{1}{p!}\sum_{r=1}^{R} c_r(\boldsymbol{v}_r \cdot \boldsymbol{z})^p = \sum_{|\beta|=p} \left(\frac{1}{\beta!} \sum_{r=1}^{R} c_r \boldsymbol{v}_r^{\beta} \right) \boldsymbol{z}^{\beta}. \tag{12}$$

We first prove that (9) implies (10). Comparing the coefficients of $\boldsymbol{z}^{\beta}$ in (9) and (12) yields (10). Conversely, to prove that (10) implies (9), we substitute (10) into (12) and obtain (9). ∎

Proposition 3.1 shows that evaluating $\mathcal{L}u(\boldsymbol{x})$ reduces to evaluating a linear combination of directional Taylor coefficients. This gives the general WDD procedure in Algorithm 1.

Algorithm 1 WDD evaluation of $\mathcal{L}u(\boldsymbol{x})$ with $\mathcal{L}$ given in (7)

Input: Function u admitting a holomorphic extension to an open neighborhood of $\boldsymbol{x}$ in $\mathbb{C}^d$; evaluation point $\boldsymbol{x} \in \Omega$; homogeneous operator $\mathcal{L}$ of order p given in (7); coefficients $c_1, c_2, \ldots, c_R$ and directions $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_R$ satisfying (9).

Output: The value of $\mathcal{L}u(\boldsymbol{x})$.

1. For $r = 1, 2, \ldots, R$, compute $T_p(\boldsymbol{x}; \boldsymbol{v}_r)$ by Taylor-mode automatic differentiation.
2. Compute $\mathcal{L}u(\boldsymbol{x}) = \sum_{r=1}^{R} c_r T_p(\boldsymbol{x}; \boldsymbol{v}_r)$.

Algorithm 1 does not specify how to obtain the power-sum representation in (9). Its existence is discussed in Section 3.2, while the minimum number of directions is related to the following concept of Waring rank.

Definition 3.1 (Complex Waring decomposition and rank). *Let $P \in \mathbb{C}[z_1, z_2, \ldots, z_d]$ be a nonzero homogeneous polynomial of degree $p \geq 1$. A complex Waring decomposition of P of length R is an identity*

$$P(\boldsymbol{z}) = \sum_{r=1}^{R} \lambda_r \ell_r(\boldsymbol{z})^p,$$

where R is a positive integer, $\lambda_r \in \mathbb{C} \setminus \{0\}$, and each ℓ_r is a nonzero complex linear form. The complex Waring rank $R_{\mathbb{C}}(P)$ is the smallest possible length of such a decomposition [5, Section 1].

By Proposition 3.1, the minimum number of directions in (8) is $R_{\mathbb{C}}\left(\sum_{|\beta|=p} a_\beta \boldsymbol{z}^\beta\right)$. For a homogeneous polynomial with rational coefficients, determining the complex Waring rank is NP-hard [26].

3.2. Computational complexity. Proposition 3.2 establishes the existence of a directional representation (8) for every operator in (7). The statement follows from Proposition 3.1 and the classical dimension bound for complex Waring rank [3, Sec. 3.1].

Proposition 3.2 (Existence of a directional representation). *For every operator $\mathcal{L}$ in (7), there exist a positive integer R, coefficients $c_1, c_2, \ldots, c_R \in \mathbb{C}$, and directions $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_R \in \mathbb{C}^d$ satisfying (9), with*

$$R \leq \binom{d+p-1}{p}. \tag{13}$$

Algorithm 1 evaluates R directional Taylor coefficients $T_p(\boldsymbol{x};\boldsymbol{v}_r)$. Taylor-mode automatic differentiation computes each coefficient at cost $O(p^2)$ [24, Sec. 4.2]. Evaluating the R coefficients and their weighted sum therefore costs

$$C_{\mathrm{WDD}} = O(Rp^2). \tag{14}$$

3.3. Monomial specialization and complex Waring rank. Fix a nonzero multi-index $\nu \in \mathbb{N}_0^d$ of order $p := |\nu| \geq 1$, and set $\mathcal{L} = \partial^\nu$. For a positive integer R, fix coefficients $c_r \in \mathbb{C}$ and directions $\boldsymbol{v}_r \in \mathbb{C}^d$, $r = 1, 2, \ldots, R$. Proposition 3.1 gives

$$\partial^\nu u(\boldsymbol{x}) = \sum_{r=1}^{R} c_r T_p(\boldsymbol{x};\boldsymbol{v}_r) \quad \Longleftrightarrow \quad p!\,\boldsymbol{z}^\nu = \sum_{r=1}^{R} c_r (\boldsymbol{v}_r \cdot \boldsymbol{z})^p, \tag{15}$$

where the left-hand identity is required to hold for every evaluation point $\boldsymbol{x}$ and every u satisfying the holomorphic-extension assumption at $\boldsymbol{x}$, and the right-hand identity holds in $\mathbb{C}[z_1, z_2, \ldots, z_d]$. To state the rank formula and an attaining decomposition, relabel the variables with positive exponents in $\boldsymbol{z}^\nu$ after a permutation of the coordinates so that

$$\boldsymbol{z}^\nu = z_0^{\nu_0} z_1^{\nu_1} \cdots z_n^{\nu_n}, \qquad 1 \leq \nu_0 \leq \nu_1 \leq \cdots \leq \nu_n, \qquad p = \sum_{j=0}^{n} \nu_j. \tag{16}$$

The rank remains unchanged when the linear forms are allowed to involve the remaining polynomial variables [5, Remark 2.3].

For each integer $m \geq 1$, let $U_m := \{\zeta \in \mathbb{C} : \zeta^m = 1\}$, and define

$$\mathcal{U}_\nu := U_{\nu_1+1} \times U_{\nu_2+1} \times \cdots \times U_{\nu_n+1}. \tag{17}$$

When $n = 0$, $\mathcal{U}_\nu$ consists of the empty tuple and has cardinality one.

The monomial rank formula is due to Carlini, Catalisano, and Geramita [5, Cor. 3.3 and Remark 2.3], while the explicit roots-of-unity decomposition is given by Buczyńska, Buczyński, and Teitler [4, Sec. 2, Eq. (8)]. The following theorem records these results in the normalization required by (15).

THEOREM 3.3 (Complex Waring rank and roots-of-unity decomposition of a monomial). *Let the variables and positive exponents of $\boldsymbol{z}^\nu$ be ordered as in* (16). *Then every R-term complex Waring decomposition of $p!\,\boldsymbol{z}^\nu$ satisfies*

$$R \geq R_{\mathbb{C}}(\boldsymbol{z}^\nu) = \prod_{j=1}^{n} (\nu_j + 1). \tag{18}$$

Moreover, the roots-of-unity identity

$$p!\,\boldsymbol{z}^\nu = \frac{\nu!}{\prod_{j=1}^{n}(\nu_j+1)} \sum_{\zeta \in \mathcal{U}_\nu} \left(\prod_{j=1}^{n} \zeta_j \right) \left(z_0 + \sum_{j=1}^{n} \zeta_j z_j \right)^p \tag{19}$$

contains exactly $\prod_{j=1}^{n}(\nu_j+1)$ terms and therefore attains the lower bound in (18).

For completeness, Appendix A.2 verifies (19) directly in the notation used here.

3.4. Minimum-direction formula and algorithm. Combining Proposition 3.1 with Theorem 3.3 gives the corresponding minimum-direction representation of a mixed partial.

COROLLARY 3.4 (Minimum-direction representation of a mixed partial). *Let the coordinates and positive entries of ν be ordered as in* (16)*. Then*

$$\partial^\nu u(\boldsymbol{x}) = \frac{\nu!}{\prod_{j=1}^n (\nu_j+1)} \sum_{\zeta \in \mathcal{U}_\nu} \left(\prod_{j=1}^n \zeta_j \right) T_p\big(\boldsymbol{x}; (1, \zeta_1, \zeta_2, \ldots, \zeta_n, 0, 0, \ldots, 0)^\top\big)\,. \quad (20)$$

Moreover, among all coefficients $c_1, c_2, \ldots, c_R \in \mathbb{C}$ and directions $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_R \in \mathbb{C}^d$ for which the left-hand identity in (15) *holds for every u satisfying the assumption in Section 2.3, the minimum possible value of R is*

$$\prod_{j=1}^n (\nu_j + 1). \quad (21)$$

Proof. Applying Proposition 3.1 to (19) gives (20). Conversely, the equivalence in (15) maps every such R-term representation, after omitting zero terms, to a Waring decomposition of $p!\,\boldsymbol{z}^\nu$ with at most R terms. The lower bound (18) gives the stated minimum, which is attained by (20). ∎

The rank formula also describes when the directional reduction is most effective. A pure derivative requires one direction, whereas a square-free derivative of order p requires 2^{p-1} directions; repeated indices can reduce this number. The minimum in Corollary 3.4 concerns one fixed monomial derivative and exact linear representations by complex order-p directional Taylor coefficients.

Algorithm 2 WDD evaluation of $\partial^\nu u(\boldsymbol{x})$ using the roots-of-unity formula (20)

Input: Function u satisfying the assumption in Section 2.3; evaluation point $\boldsymbol{x} \in \Omega$; multi-index $\nu \in \mathbb{N}_0^d$ with $p = |\nu| \geq 1$.
Output: The value of $\partial^\nu u(\boldsymbol{x})$.

1. Order the coordinates associated with the positive entries of ν as in (16), retaining their positions in the original coordinate system.
2. Form the roots-of-unity index set $\mathcal{U}_\nu$ defined in (17).
3. For each $\zeta = (\zeta_1, \zeta_2, \ldots, \zeta_n) \in \mathcal{U}_\nu$, construct $\boldsymbol{v}_\zeta \in \mathbb{C}^d$ with entries $(1, \zeta_1, \zeta_2, \ldots, \zeta_n)$ in these ordered coordinate positions and zero entries in all remaining positions.
4. Compute $T_p(\boldsymbol{x}; \boldsymbol{v}_\zeta)$ for each $\zeta \in \mathcal{U}_\nu$ by Taylor-mode automatic differentiation.
5. Evaluate the weighted sum in (20) to obtain $\partial^\nu u(\boldsymbol{x})$.

4. Numerical experiments. We evaluate the proposed Waring decomposition of derivatives (WDD) method in two complementary settings. First, we measure the evaluation time of individual high-order partial derivatives across various orders and multi-index patterns, using the minimum-direction complex Waring representation in Corollary 3.4. Second, we assess PINN training on three nonlinear differential equations: a one-dimensional KdV equation with a gradient-enhanced objective, a two-dimensional KdV equation with mixed derivatives, and a KP-II equation. For the PDE problems, we apply Algorithm 1 to real power-sum identities, allowing Taylor coefficients to be reused across different derivative terms. The compared methods compute the same mathematical derivatives. The KdV experiments compare WDD

with nested coordinate Jacobian–vector products (JVPs), and the KP-II experiment compares WDD with the STDE++ construction, under identical loss objectives and matched computational conditions.

4.1. Individual high-order partial derivatives. We compare WDD with a centered-polarization implementation of the directional Taylor identity reviewed in STDE++ [25, Section 6.1, Eq. (25)] for individual partial derivatives $\partial_{i_1}\partial_{i_2}\cdots\partial_{i_p}u$. We set the batch size to $B = 1000$. The test network has 16 inputs and three hidden layers of width 128 with tanh activations, and both methods use identical weights and inputs and the same Taylor propagation routine on a Tesla V100. Here R is the complex Waring rank. The order-p coefficient counts in Table 4.1 are per input point, after conjugate reuse in WDD and merging redundant directions in STDE++. Times are the mean and sample standard deviation of three seed medians (30 synchronized calls after 10 warm-ups per seed), excluding direction preparation and parameter backpropagation.

TABLE 4.1

Ranks, Taylor coefficient counts, and evaluation times on a Tesla V100 for WDD and STDE++ (Section 6.1). Speedups are mean paired ratios of STDE++ to WDD times.

Target	p	R	Taylor coefficients		Time (ms)		Speedup
			STDE++	WDD	STDE++	WDD	
$\partial_1\partial_2\partial_3\partial_4\partial_5\partial_6 u$	6	32	32	32	29.605 ± 0.186	29.600 ± 0.145	1.00×
$\partial_1^2\partial_2^2\partial_3^2\partial_4^2 u$	8	27	40	14	60.533 ± 0.028	45.456 ± 0.029	1.33×
$\partial_1^3\partial_2^3\partial_3^3 u$	9	16	28	10	51.341 ± 0.007	39.932 ± 0.019	1.29×
$\partial_1^4\partial_2^4\partial_3^4 u$	12	25	49	13	153.037 ± 0.025	85.421 ± 0.009	1.79×
$\partial_1^5\partial_2^5\partial_3^5 u$	15	36	100	20	479.972 ± 0.006	195.631 ± 0.026	2.45×

4.2. PINN training on nonlinear differential equations. For all PDE benchmarks, u_θ is a fully connected network with four hidden layers of width 128, tanh activations, and a scalar linear output. We train with Adam [16] using a learning rate of 10^{-5} for the KdV problems and 10^{-4} for KP-II. At each update we independently resample 400 interior space–time points and points on each initial and spatial boundary face: 128 points per face in 1D and 256 per face in 2D. Let $\mathcal{T}_r$, $\mathcal{T}_0$, and $\mathcal{T}_b$ be the interior, initial, and combined spatial boundary point sets. Conditions on the same face use the same points. For each of five seeds, the paired methods share the initialization, loss weights, and samples at each common update index. Both methods are trained for 25,000 updates for one-dimensional KdV and 20,000 updates for two-dimensional KdV and KP-II. The KP-II averages use seeds 20260936–20260940. In the polynomial identities below, coordinate letters also denote the corresponding formal variables. For a test set D, we record the relative L^2 error

$$\mathrm{RE} := \frac{\left(\sum_{\boldsymbol{x}\in D} |u_\theta(\boldsymbol{x}) - u_\star(\boldsymbol{x})|^2\right)^{1/2}}{\left(\sum_{\boldsymbol{x}\in D} |u_\star(\boldsymbol{x})|^2\right)^{1/2}}, \tag{22}$$

evaluated on 10,000 uniform space–time test points. The test set is fixed, independent of training, and shared by paired methods. Tables report training times and RE as means and sample standard deviations across seeds. Training speedups are means of paired baseline-to-WDD training-time ratios. Figure curves show mean errors against mean training times at common update indices, with shading extending one sample standard deviation upward.

4.2.1. Gradient-enhanced PINN for the KdV equation. This example tests the evaluation of fourth-order derivatives arising from a gradient-enhanced PINN objective. We solve

$$\begin{cases} u_t + uu_x + 0.0025u_{xxx} = 0, & x \in (-1,1), \quad t \in (0,1), \\ u(x,0) = u_\star(x,0), & x \in [-1,1], \\ u(-1,t) = u_\star(-1,t), & t \in (0,1), \\ u(1,t) = u_\star(1,t), \quad \partial_x u(1,t) = \partial_x u_\star(1,t), & t \in (0,1). \end{cases} \tag{23}$$

The exact solution is the traveling solitary wave

$$u_\star(x,t) = 0.75\ \mathrm{sech}^2\big(5(x + 0.5 - 0.25t)\big). \tag{24}$$

Define the PDE residual by

$$r_\theta(x,t) := (u_\theta)_t + u_\theta(u_\theta)_x + 0.0025(u_\theta)_{xxx},$$

and the initial and boundary penalty by

$$\begin{aligned} \mathcal{J}_b(\theta) = \frac{1}{|\mathcal{T}_0|} \sum_{(x,0)\in\mathcal{T}_0} \big[u_\theta(x,0) - u_\star(x,0)\big]^2 + \frac{1}{128} \sum_{(-1,t)\in\mathcal{T}_b} \big[u_\theta(-1,t) - u_\star(-1,t)\big]^2 \\ + \frac{1}{128} \sum_{(1,t)\in\mathcal{T}_b} \Big\{ \big[u_\theta(1,t) - u_\star(1,t)\big]^2 + \big[\partial_x u_\theta(1,t) - \partial_x u_\star(1,t)\big]^2 \Big\}. \end{aligned}$$

The objective is

$$\mathcal{J}(\theta) = \frac{1}{|\mathcal{T}_r|} \sum_{(x,t)\in\mathcal{T}_r} \Big[r_\theta^2 + \lambda_1 \big((\partial_x r_\theta)^2 + (\partial_t r_\theta)^2 \big) \Big] + \lambda_2 \mathcal{J}_b(\theta), \tag{25}$$

where $\lambda_1 = 10^{-3}$ and $\lambda_2 = 10$ are the loss weights. To compute u_{xxxt}, notice that the mixed symbol $x^3 t$ admits the decomposition

$$x^3 t = \tfrac{1}{6}\big[(x+t)^4 - (x-t)^4\big] - \tfrac{1}{48}\big[(x+2t)^4 - (x-2t)^4\big]. \tag{26}$$

Applying Proposition 3.1 to (26) and x^4 gives

$$\begin{aligned} u_{xxxx} &= 24 T_4(\boldsymbol{x}; (1,0)^\top), \\ u_{xxxt} &= 4\big[T_4(\boldsymbol{x}; (1,1)^\top) - T_4(\boldsymbol{x}; (1,-1)^\top)\big] - \tfrac{1}{2}\big[T_4(\boldsymbol{x}; (1,2)^\top) - T_4(\boldsymbol{x}; (1,-2)^\top)\big]. \end{aligned} \tag{27}$$

These five directions also provide the lower-order coefficients required by the objective. Table 4.2 and Figure 4.1 show that the WDD method attains comparable mean final errors with a 7.82× training speedup.

TABLE 4.2
Numerical results for the one-dimensional KdV equation (23).

Method	Training time (s)	RE
Nested JVP	8779.2 ± 565.2	$4.955 \times 10^{-3} \pm 1.297 \times 10^{-3}$
WDD	1123.8 ± 15.7	$4.955 \times 10^{-3} \pm 1.297 \times 10^{-3}$

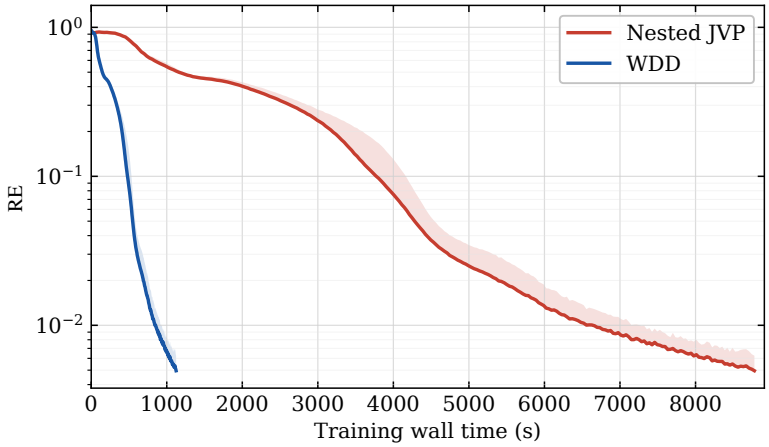


FIG. 4.1. RE *during training for the one-dimensional KdV equation* (23).

4.2.2. Mixed-derivative PINN for a two-dimensional KdV equation. This example combines the spatial mixed derivative u_{xxxy} with the space–time derivative u_{ty}. We consider the KdV-type equation in [24, Eq. (20)],

$$\begin{cases} u_{ty} + u_{xxxy} + 3(u_x u_y)_x - u_{xx} + 2u_{yy} = 0, & (x,y) \in (-1,1)^2, \quad t \in (0,1), \\ u(x,y,0) = u_\star(x,y,0), & (x,y) \in [-1,1]^2, \\ u(-1,y,t) = u_\star(-1,y,t), & y \in [-1,1], \quad t \in (0,1), \\ \quad u(1,y,t) = u_\star(1,y,t), & \\ \partial_x u(1,y,t) = \partial_x u_\star(1,y,t), & y \in [-1,1], \quad t \in (0,1), \\ \quad u(x,-1,t) = u_\star(x,-1,t), & \\ \partial_y u(x,-1,t) = \partial_y u_\star(x,-1,t), & x \in [-1,1], \quad t \in (0,1). \end{cases} \tag{28}$$

The exact oblique-kink solution is

$$u_\star(x,y,t) = \tanh\big((x+y)/2 - t\big). \tag{29}$$

Only the PDE is taken from the cited work; the domain, exact solution, and initial–boundary conditions specify the present benchmark. Define the PDE residual by

$$r_\theta(x,y,t) := (u_\theta)_{ty} + (u_\theta)_{xxxy} + 3\partial_x\big[(u_\theta)_x (u_\theta)_y\big] - (u_\theta)_{xx} + 2(u_\theta)_{yy},$$

and the initial and boundary penalty by

$$\begin{aligned} \mathcal{J}_b(\theta) = {} & \frac{1}{|\mathcal{T}_0|} \sum_{(x,y,0)\in\mathcal{T}_0} \big[u_\theta(x,y,0) - u_\star(x,y,0)\big]^2 \\ & + \frac{1}{256} \sum_{(-1,y,t)\in\mathcal{T}_b} \big[u_\theta(-1,y,t) - u_\star(-1,y,t)\big]^2 \\ & + \frac{1}{256} \sum_{(1,y,t)\in\mathcal{T}_b} \Big\{ \big|(u_\theta - u_\star)(1,y,t)\big|^2 + \big|\partial_x(u_\theta - u_\star)(1,y,t)\big|^2 \Big\} \\ & + \frac{1}{256} \sum_{(x,-1,t)\in\mathcal{T}_b} \Big\{ \big|(u_\theta - u_\star)(x,-1,t)\big|^2 + \big|\partial_y(u_\theta - u_\star)(x,-1,t)\big|^2 \Big\}. \end{aligned}$$

The objective is

$$\mathcal{J}_{2D}(\theta) = \frac{1}{|\mathcal{T}_r|} \sum_{(x,y,t)\in\mathcal{T}_r} r_\theta(x,y,t)^2 + \lambda_2 \mathcal{J}_b(\theta). \tag{30}$$

Using (26) with t replaced by y, together with

$$yt = \tfrac{1}{4}\big[(y+t)^2 - (y-t)^2\big], \tag{31}$$

Proposition 3.1 gives, with $\boldsymbol{x} = (x,y,t)^\top$ the network input point,

$$\begin{aligned} u_{xxxy} &= 4\big[T_4(\boldsymbol{x};(1,1,0)^\top) - T_4(\boldsymbol{x};(1,-1,0)^\top)\big] \\ &\quad - \tfrac{1}{2}\big[T_4(\boldsymbol{x};(1,2,0)^\top) - T_4(\boldsymbol{x};(1,-2,0)^\top)\big], \\ u_{ty} &= \tfrac{1}{2}\big[T_2(\boldsymbol{x};(0,1,1)^\top) - T_2(\boldsymbol{x};(0,1,-1)^\top)\big]. \end{aligned} \tag{32}$$

Algorithm 1 evaluates Taylor expansions along six real directions. Their lower-order coefficients also recover $u_x, u_y, u_{xx}, u_{xy}, u_{yy}$. The results in Table 4.3 and Figure 4.2 show that the WDD method attains comparable mean final errors with a 5.22× training speedup.

TABLE 4.3
Numerical results for the two-dimensional KdV equation (28).

Method	Training time (s)	RE
Nested JVP	6354.1 ± 257.2	$1.020 \times 10^{-3} \pm 2.598 \times 10^{-4}$
WDD	1217.1 ± 10.9	$1.020 \times 10^{-3} \pm 2.598 \times 10^{-4}$

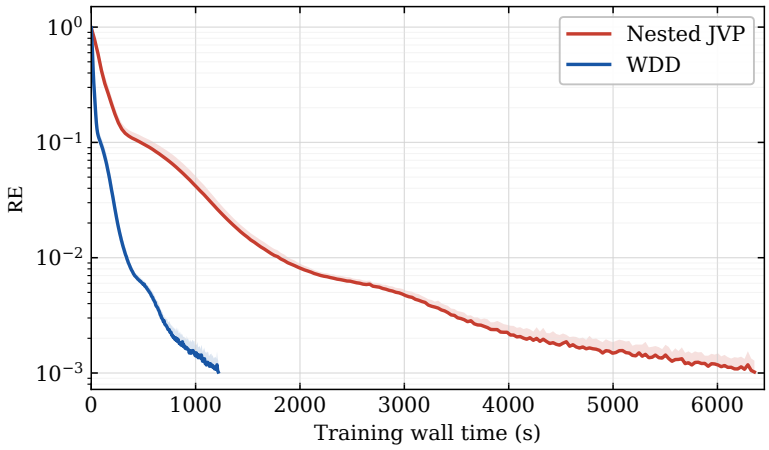


FIG. 4.2. RE *during training for the two-dimensional KdV equation* (28).

4.2.3. KP-II equation. This example compares WDD with the KP derivative construction used in the STDE++ implementation. We consider the line-soliton problem in [22, Eqs. (3.7)–(3.9)], with $\Omega = (-3,3)^2$,

$$\begin{cases} u_{tx} + 6(u_x^2 + u u_{xx}) + u_{xxxx} + 3u_{yy} = 0, & (x,y)\in\Omega, \quad t\in(-\tfrac{1}{2},\tfrac{1}{2}), \\ u(x,y,-\tfrac{1}{2}) = u_\star(x,y,-\tfrac{1}{2}), & (x,y)\in\overline{\Omega}, \\ u(x,y,t) = u_\star(x,y,t), & (x,y)\in\partial\Omega, \quad t\in(-\tfrac{1}{2},\tfrac{1}{2}). \end{cases} \tag{33}$$

The exact solution is

$$u_\star(x,y,t) = \frac{18e^{3x+6y-63t}}{(1+e^{3x+6y-63t})^2}. \tag{34}$$

Define the PDE residual by

$$r_\theta := (u_\theta)_{tx} + 6\big[(u_\theta)_x^2 + u_\theta(u_\theta)_{xx}\big] + (u_\theta)_{xxxx} + 3(u_\theta)_{yy}.$$

Let $\mathcal{F}$ denote the four spatial boundary faces. The objective is

$$\begin{aligned}\mathcal{J}_{\mathrm{KP}}(\theta) = \frac{1}{|\mathcal{T}_r|}\sum_{\boldsymbol{x}\in\mathcal{T}_r} r_\theta(\boldsymbol{x})^2 + \lambda_2\Bigg\{ &\frac{1}{|\mathcal{T}_0|}\sum_{\boldsymbol{x}\in\mathcal{T}_0} |u_\theta(\boldsymbol{x}) - u_\star(\boldsymbol{x})|^2 \\ &+ \sum_{F\in\mathcal{F}} \frac{1}{256} \sum_{\substack{\boldsymbol{x}\in\mathcal{T}_b \\ (x,y)\in F}} |u_\theta(\boldsymbol{x}) - u_\star(\boldsymbol{x})|^2\Bigg\}, \qquad \lambda_2 = 10. \end{aligned} \tag{35}$$

For $\boldsymbol{x} = (x,y,t)^\top$, the power-sum identity $xt = \frac{1}{4}[(x+t)^2 - (x-t)^2]$ gives

$$\begin{aligned}(u_\theta)_{xxxx} &= 24T_4(\boldsymbol{x}; (1,0,0)^\top), \\ (u_\theta)_{tx} &= \tfrac{1}{2}\big[T_2(\boldsymbol{x}; (1,0,1)^\top) - T_2(\boldsymbol{x}; (1,0,-1)^\top)\big], \\ (u_\theta)_{yy} &= 2T_2(\boldsymbol{x}; (0,1,0)^\top).\end{aligned} \tag{36}$$

The order-four expansion also supplies u_θ, $(u_\theta)_x$, and $(u_\theta)_{xx}$. The STDE++ baseline recovers $(u_\theta)_{tx}$ from the fifth-order expansion along $\boldsymbol{x}+\tau^2(1,0,0)^\top/2+\tau^3(0,0,1)^\top/6$, together with coordinate Taylor expansions for the remaining terms, following its published implementation [25]. Table 4.4 and Figure 4.3 show final mean errors of 3.726×10^{-2} and 4.052×10^{-2} for WDD and STDE++, respectively, with a 1.32× training speedup.

TABLE 4.4
Numerical results for the KP-II equation (33).

Method	Training time (s)	RE
WDD	913.1 ± 37.3	$3.726 \times 10^{-2} \pm 6.034 \times 10^{-3}$
STDE++	1205.6 ± 29.5	$4.052 \times 10^{-2} \pm 7.050 \times 10^{-3}$

Across the PDE experiments, the WDD method attains comparable mean final errors at equal update counts. It requires less training time, with both interior and constraint training points resampled at every update.

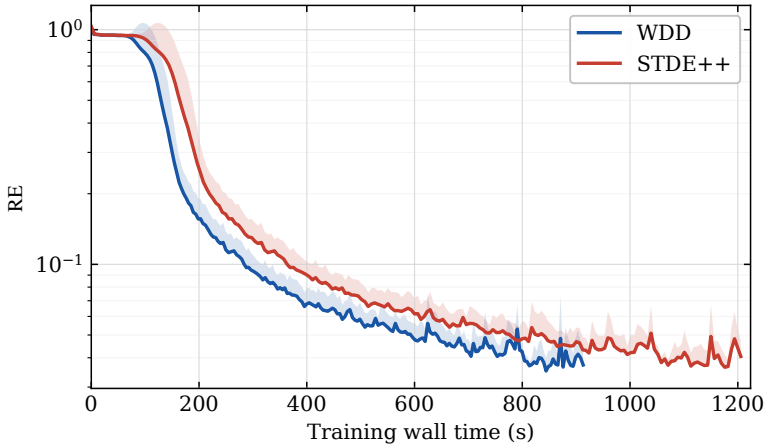


FIG. 4.3. RE *during training for the KP-II equation* (33).

5. Conclusion. Within the Taylor-mode framework used by STDE [24], WDD constructs exact directional representations of high-order derivatives from polynomial symbols. Classical Waring-rank theory and the roots-of-unity decomposition give complex directional Taylor representations of minimum length for mixed partials. The WDD method evaluates these representations by batched Taylor propagation, while real power-sum identities allow Taylor coefficients to be reused across PDE derivative terms. The experiments show faster individual-partial evaluation for several tested patterns and shorter PINN training time than nested coordinate JVPs for KdV and the STDE++ construction for KP-II at equal update counts. With interior and constraint points resampled at every update, the WDD method attains comparable mean final errors for the tested KdV and KP-II problems.

Appendix A. Algebraic verification.

A.1. Roots-of-unity filter. LEMMA A.1. *For every integer $m \geq 1$ and every integer k,*

$$\sum_{\zeta \in U_m} \zeta^k = \begin{cases} m, & m \mid k, \\ 0, & m \nmid k. \end{cases} \tag{37}$$

Proof. Let $\omega := \exp(2\pi \mathrm{i}/m)$, so that every element of U_m is ω^ℓ for exactly one $\ell \in \{0, 1, \ldots, m-1\}$ and

$$\sum_{\zeta \in U_m} \zeta^k = \sum_{\ell=0}^{m-1} \left(\omega^k\right)^\ell.$$

If $m \mid k$, then $\omega^k = 1$ and the sum equals m. If $m \nmid k$, then $\omega^k \neq 1$, and the finite geometric series gives

$$\sum_{\ell=0}^{m-1} \left(\omega^k\right)^\ell = \frac{1 - \omega^{km}}{1 - \omega^k} = 0,$$

since $\omega^m = 1$. ∎

A.2. Verification of the explicit monomial decomposition. For completeness, we verify the decomposition (19) of Buczyńska, Buczyński, and Teitler [4, Sec. 2, Eq. (8)] in the notation used here. For $\beta = (\beta_0, \beta_1, \dots, \beta_n) \in \mathbb{N}_0^{n+1}$ with $|\beta| = p$, the coefficient of $\boldsymbol{z}^\beta$ on the right-hand side of (19) is

$$\frac{\nu!}{\prod_{j=1}^n (\nu_j + 1)} \binom{p}{\beta} \prod_{j=1}^n \left(\sum_{\zeta_j \in U_{\nu_j+1}} \zeta_j^{\beta_j+1} \right). \tag{38}$$

By Lemma A.1, the product of sums in (38) can be nonzero only if $\beta_j + 1$ is a positive integer multiple of $\nu_j + 1$ for every $j = 1, 2, \dots, n$. Hence each β_j is either ν_j or at least $2\nu_j + 1$. If $\beta_j \geq 2\nu_j + 1$ for some $j \geq 1$, then $\beta_k \geq \nu_k$ for every $k = 1, 2, \dots, n$, and therefore

$$\beta_0 = p - \sum_{k=1}^n \beta_k \leq \nu_0 - \nu_j - 1 < 0,$$

which contradicts $\beta_0 \geq 0$. Thus $\beta_j = \nu_j$ for $j = 1, 2, \dots, n$, and $|\beta| = p$ gives $\beta_0 = \nu_0$. Consequently, the coefficient in (38) vanishes unless $\beta_j = \nu_j$ for every $j = 0, 1, \dots, n$. In that case, the coefficient equals

$$\frac{\nu!}{\prod_{j=1}^n (\nu_j + 1)} \frac{p!}{\nu!} \prod_{j=1}^n (\nu_j + 1) = p!.$$

This proves (19).

REFERENCES

[1] Atilim Gunes Baydin, Barak A. Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind, *Automatic differentiation in machine learning: a survey*, Journal of Machine Learning Research, 18 (2018), pp. 1–43.

[2] Jesse Bettencourt, Matthew J. Johnson, and David Duvenaud, *Taylor-mode automatic differentiation for higher-order derivatives in JAX*, in NeurIPS Workshop on Program Transformations for Machine Learning, 2019.

[3] Grigoriy Blekherman and Zach Teitler, *On maximum, typical and generic ranks*, Mathematische Annalen, 362 (2015), pp. 1021–1031.

[4] Weronika Buczyńska, Jarosław Buczyński, and Zach Teitler, *Waring decompositions of monomials*, Journal of Algebra, 378 (2013), pp. 45–57.

[5] Enrico Carlini, Maria Virginia Catalisano, and Anthony V. Geramita, *The solution to the Waring problem for monomials and the sum of coprime monomials*, Journal of Algebra, 370 (2012), pp. 5–14.

[6] Pao-Hsiung Chiu, Jian Cheng Wong, Chinchun Ooi, My Ha Dao, and Yew-Soon Ong, *CAN-PINN: A fast physics-informed neural network based on coupled-automatic–numerical differentiation method*, Computer Methods in Applied Mechanics and Engineering, 395 (2022), p. 114909.

[7] Bengt Fornberg, *Generation of finite difference formulas on arbitrarily spaced grids*, Mathematics of Computation, 51 (1988), pp. 699–706.

[8] Andreas Griewank, Jean Utke, and Andrea Walther, *Evaluating higher derivative tensors by forward propagation of univariate Taylor series*, Mathematics of Computation, 69 (2000), pp. 1117–1130.

[9] Andreas Griewank and Andrea Walther, *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, Society for Industrial and Applied Mathematics, Philadelphia, PA, 2nd ed., 2008.

[10] Di He, Shanda Li, Wenlei Shi, Xiaotian Gao, Jia Zhang, Jiang Bian, Liwei Wang, and Tie-Yan Liu, *Learning physics-informed neural networks without stacked back-propagation*, in Proceedings of the 26th International Conference on Artificial Intelligence and Statistics, vol. 206 of Proceedings of Machine Learning Research, PMLR, 2023, pp. 3034–3047.

[11] Zheyuan Hu, Zekun Shi, George Em Karniadakis, and Kenji Kawaguchi, *Hutchinson trace estimation for high-dimensional and high-order physics-informed neural networks*, Computer Methods in Applied Mechanics and Engineering, 424 (2024), p. 116883.

[12] ———, *Bias-variance trade-off in physics-informed neural networks with randomized smoothing for high-dimensional PDEs*, Journal of Computational Physics, 521 (2025), p. 113524.

[13] Zheyuan Hu, Khemraj Shukla, George Em Karniadakis, and Kenji Kawaguchi, *Tackling the curse of dimensionality with physics-informed neural networks*, Neural Networks, 176 (2024), p. 106369.

[14] Michael F. Hutchinson, *A stochastic estimator of the trace of the influence matrix for Laplacian smoothing splines*, Communications in Statistics: Simulation and Computation, 18 (1989), pp. 1059–1076.

[15] George Em Karniadakis, Ioannis G. Kevrekidis, Lu Lu, Paris Perdikaris, Sifan Wang, and Liu Yang, *Physics-informed machine learning*, Nature Reviews Physics, 3 (2021), pp. 422–440.

[16] Diederik P. Kingma and Jimmy Ba, *Adam: A method for stochastic optimization*, in International Conference on Learning Representations (ICLR), 2015.

[17] Isaac E. Lagaris, Aristidis Likas, and Dimitrios I. Fotiadis, *Artificial neural networks for solving ordinary and partial differential equations*, IEEE Transactions on Neural Networks, 9 (1998), pp. 987–1000.

[18] Ruichen Li, Haotian Ye, Du Jiang, Xuelan Wen, Chuwei Wang, Zhe Li, Xiang Li, Di He, Ji Chen, Weiluo Ren, and Liwei Wang, *A computational framework for neural network-based variational Monte Carlo with Forward Laplacian*, Nature Machine Intelligence, 6 (2024), pp. 209–219.

[19] Lynn H. Loomis and Shlomo Sternberg, *Advanced Calculus*, Jones and Bartlett, Boston, MA, revised ed., 1990.

[20] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala, *PyTorch: An imperative style, high-performance deep learning library*, in Advances in Neural Information Processing Systems, vol. 32, 2019.

[21] Barak A. Pearlmutter, *Fast exact multiplication by the Hessian*, Neural Computation, 6 (1994), pp. 147–160.

[22] Juncai Pu and Yong Chen, *Lax pairs informed neural networks solving integrable systems*, arXiv preprint arXiv:2401.04982, (2024).

[23] Maziar Raissi, Paris Perdikaris, and George E. Karniadakis, *Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations*, Journal of Computational Physics, 378 (2019), pp. 686–707.

[24] Zekun Shi, Zheyuan Hu, Min Lin, and Kenji Kawaguchi, *Stochastic Taylor derivative estimator: Efficient amortization for arbitrary differential operators*, Advances in Neural Information Processing Systems (NeurIPS), 37 (2024), pp. 122316–122353.

[25] ———, *STDE++: Polynomial-time amortization for linear differential operators*, Journal of Machine Learning Research, 27 (2026), pp. 1–50.

[26] Yaroslav Shitov, *How hard is the tensor rank?*, 2016. arXiv:1611.01559.

[27] Justin Sirignano and Konstantinos Spiliopoulos, *DGM: A deep learning algorithm for solving partial differential equations*, Journal of Computational Physics, 375 (2018), pp. 1339–1364.

[28] Charles M. Stein, *Estimation of the mean of a multivariate normal distribution*, The Annals of Statistics, 9 (1981), pp. 1135–1151.